

ОСР — Архитектура и сайзинг Self-hosted Edition

Omnichannel communication platform

Exported on 09/19/2026

Table of Contents

Содержание	8
0. Краткий справочник: выбор конфигурации	9
Pilot – до 1 000 диалогов/мес	9
Start – до 10 000 диалогов/мес	10
Standard – 10 000 – 100 000 диалогов/мес	11
Medium – 100 000 – 200 000 диалогов/мес	12
Large – 200 000 – 300 000 диалогов/мес	13
1. Обзор	15
Назначение	15
Требования к ПО	15
Поддерживаемые платформы виртуализации	15
2. Архитектура компонентов	16
3. Список компонентов и контейнеров	17
Основные компоненты	17
Мониторинг	18
Опциональные компоненты	18
Потребление RAM по компонентам (типичное)	19
4. Сетевая конфигурация	20
4.1. Домены	20
4.2. DNS-записи	21
4.3. Таблица портов	21
4.4. IP-адреса по конфигурациям	22
4.5. Firewall (UFW)	23
Исходящий трафик	23
5. SSL-сертификаты	25
Варианты	25
Требования	25

Let's Encrypt с Certbot.....	25
6. Конфигурации сайзинга	26
Базовые метрики для расчёта	26
Формула расчёта хранения.....	26
Конфигурация Pilot — до 1 000 диалогов/мес	26
Конфигурация Start — до 10 000 диалогов/мес.....	27
Конфигурация Standard — 10 000 – 100 000 диалогов/мес.....	28
Конфигурация Medium — 100 000 – 200 000 диалогов/мес	29
Конфигурация Large — 200 000 – 300 000 диалогов/мес	30
7. Дисковое пространство — детальный расчёт.....	32
7.1. PostgreSQL data (SSD обязательно!)	32
7.2. Логи (HDD допустим)	32
7.3. S3 Storage (вложения, файлы)	33
7.4. Docker images + OS	33
7.5. Сводная таблица.....	34
8. Архитектура масштабирования	35
Level 1: All-in-one (1 сервер)	35
Level 2: App + DB (2 сервера)	35
Level 3: App + DB + S3/Logs (3–4 сервера)	35
Level 4: Кластер (5–6 серверов).....	35
9. Взаимодействие слоёв	37
9.1. Типичные потоки данных.....	37
10. Резервное копирование	38
Стратегия	38
Размер бэкапов	38
RPO / RTO	39
Пример cron для pg_dump	39
Куда складывать.....	39
11. Обновление системы	40

Процедура обновления	40
Downtime	40
Рекомендации.....	40
12. Внешние зависимости и SMTP	42
SMTP (обязательно для production)	42
Внешние API-зависимости	42
13. Чек-лист для DevOps	44
13.1. Пререквизиты.....	44
13.2. Порядок установки.....	44
13.3. Валидация после установки	44
13.4. Мониторинг (рекомендации)	45
Приложение А: Сводная таблица конфигураций	47
План развёртывания платформы OCP	49
Общее описание	49
Стадия 1: Планирование и подготовка инфраструктуры.....	49
1.1 Проектирование сетевой архитектуры.....	49
1.2 Выбор архитектуры развёртывания	49
1.3 Планирование доменов	50
Мастер-домен SBC (критически важно)	50
Клиентские домены SBC	50
Wildcard-сертификат (обязательно)	50
Обязательные домены платформы	51
Дополнительные домены (при необходимости)	51
Процедура утверждения доменов.....	52
1.4 Заказ лицензий SBC.....	52
Количественные параметры лицензии (DQL)	52
Пример конфигурации лицензии	54
Особенности распределения лицензий	54
Действия по заказу лицензии.....	55

Стадия 2: Установка базовых компонентов	55
2.1 Установка PostgreSQL	55
2.2 Установка Nginx.....	56
Стадия 3: Установка хранилища и мониторинга	57
3.1 Установка S3-хранилища (MinIO)	57
3.2 Установка системы мониторинга.....	57
Стадия 4: Установка и настройка SBC	57
4.1 Развертывание SBC	57
4.2 Первоначальная настройка SBC	58
4.3 Установка патчей и настройка логотипов	58
4.4 Настройка записи разговоров и установка Nginx на SBC	58
4.5 Тестирование SBC.....	59
4.6 Установка Apache Superset.....	60
Стадия 5: Установка дополнительных сервисов (опционально)	60
5.1 Установка Storage API сервиса	60
Стадия 6: Развертывание OCP	61
6.1 Подготовка компонентов OCP	61
6.2 Установка OCP	61
6.3 Переменные окружения OCP (.env).....	61
База данных PostgreSQL (обязательно)	61
Хранилище S3/MinIO (обязательно)	61
Интеграция с SBC (обязательно)	62
Общие настройки системы	62
Система очередей и лицензирование	62
Суперадминистратор (обязательно)	62
Медиа-сервер (при использовании видео)	62
Запись экранов (при необходимости)	63
Интеграция с Apache Superset (обязательно).....	63
Интеграция с Active Directory (опционально).....	63

Фронтенд в режиме разработки (только для разработки).....	63
6.4 Настройка интеграций	63
6.5 Настройка мониторинга OCP.....	64
Стадия 7: Интеграция с внешними системами.....	64
7.1 Интеграция с Active Directory (при необходимости)	64
7.3 Настройка видеозаписи (при необходимости)	64
Стадия 8: Финальная настройка и тестирование	65
8.1 Создание административных пользователей	65
8.2 Комплексное тестирование.....	65
8.3 Оптимизация производительности	65
Порядок установки компонентов (краткая схема)	65
Критически важные моменты.....	66

Создатель: Горлач Владимир Валерьевич

Версия документа: 2.0

Дата: Февраль 2026

Аудитория: DevOps / System Administrators

Назначение: Планирование серверной инфраструктуры для развёртывания Self-hosted Edition

Содержание

- [0. Краткий справочник: выбор конфигурации \(see page 0\)](#)
 - [1. Обзор \(see page 0\)](#)
 - [2. Архитектура компонентов \(see page 0\)](#)
 - [3. Список Docker-контейнеров \(see page 0\)](#)
 - [4. Сетевая конфигурация \(see page 0\)](#)
 - [5. SSL-сертификаты \(see page 0\)](#)
 - [6. Конфигурации сайзинга \(see page 0\)](#)
 - [7. Дисковое пространство — детальный расчёт \(see page 0\)](#)
 - [8. Архитектура масштабирования \(see page 0\)](#)
 - [9. Взаимодействие слоёв \(see page 0\)](#)
 - [10. Резервное копирование \(see page 0\)](#)
 - [11. Обновление системы \(see page 0\)](#)
 - [12. Внешние зависимости и SMTP \(see page 0\)](#)
 - [13. Чек-лист для DevOps \(see page 0\)](#)
 - [Приложение А: Сводная таблица конфигураций \(see page 0\)](#)
 - [План развёртывания платформы ОСР \(see page 0\)](#)
-

0. Краткий справочник: выбор конфигурации

Самодостаточные карточки для быстрого выбора. Подробности — в разделах 6–8.

Pilot — до 1 000 диалогов/мес

- **Архитектура:** All-in-one (Level 1)
- **Серверов:** 1
- **IP:** 1 публичный
- **Домены:** ocp.company.com, studio.company.com, reports.company.com и др.

Роль	vCPU	RAM	SSD	HDD
All-in-one	6–8	12–16 GB	100 GB	—

Минимум для запуска: 6 vCPU / 12 GB RAM / 100 GB SSD. Рекомендуется 8 vCPU / 16 GB RAM для стабильной работы.

Нагрузка:

Метрика	Значение
Пиковый RPS	~2–5
WebSocket соединений	10–30
Операторов онлайн	до 5

Хранение (1 год):

Компонент	Объём	Тип
PostgreSQL	~0.35 GB	SSD
Логи	~0.5 GB	SSD
Файлы/S3	~0.5 GB	SSD

HA: Нет*Переход на Start при >800 диалогов/мес или >80% CPU sustained.*

Start — до 10 000 диалогов/мес

- **Архитектура:** All-in-one (Level 1)
- **Серверов:** 1
- **IP:** 1 публичный
- **Домены:** ocp.company.com, studio.company.com, reports.company.com и др.

Роль	vCPU	RAM	SSD	HDD
All-in-one	8	16 GB	100 GB	50 GB

Нагрузка:

Метрика	Значение
Пиковый RPS	~15
WebSocket соединений	50–100
Операторов онлайн	до 20

Хранение (1 год):

Компонент	Объём	Тип
PostgreSQL	~3 GB	SSD
Логи	~5 GB	HDD
Файлы/S3	~5 GB	HDD

HA: Нет*Переход на Standard при >8 000 диалогов/мес или >80% CPU sustained.*

Standard – 10 000 – 100 000 диалогов/мес

- **Архитектура:** App + DB (Level 2)
- **Серверов:** 2
- **IP:** 1 публичный + 1 приватный
- **Домены:** ocp.company.com, studio.company.com, reports.company.com и др.

Роль	vCPU	RAM	SSD	HDD
App	8	32 GB	60 GB	50 GB
DB	8	32 GB	300 GB	100 GB
Суммарно	16	64 GB	—	—

Нагрузка:

Метрика	Значение
Пиковый RPS	~150
WebSocket соединений	500–2 000
Операторов онлайн	до 200

Хранение (1 год):

Компонент	Объём	Тип
PostgreSQL	~35 GB	SSD
Логи	~50 GB	HDD
Файлы/S3	~50 GB	HDD

HA: Опционально (DB Replica)

Переход на Medium при >80 000 диалогов/мес или >70% RAM на DB-сервере sustained.

Medium – 100 000 – 200 000 диалогов/мес

- **Архитектура:** App + DB + S3/Logs (Level 3)
- **Серверов:** 3–4 (+ опционально DB Replica)
- **IP:** 1 публичный + 2–3 частных
- **Домены:** ocp.company.com, studio.company.com, reports.company.com и др.

Роль	vCPU	RAM	SSD	HDD
App	8	32 GB	60 GB	50 GB
DB Primary	12	48 GB	500 GB	50 GB
S3/Logs	4	8 GB	50 GB	300 GB
DB Replica ¹	12	48 GB	500 GB	50 GB
Суммарно (без реплики)	24	88 GB	—	—
Суммарно (с репликой)	36	136 GB	—	—

¹ DB Replica опциональна, но рекомендуется при наличии требований к отказоустойчивости.

Нагрузка:

Метрика	Значение
Пиковый RPS	~300
WebSocket соединений	2 000–5 000
Операторов онлайн	до 500

Хранение (1 год):

Компонент	Объём	Тип
PostgreSQL	~69 GB	SSD

Логи	~100 GB	HDD
Файлы/S3	~100 GB	HDD

НА: Рекомендуется

Переход на Large при >170 000 диалогов/мес или >75% CPU на App-сервере sustained.

Large – 200 000 – 300 000 диалогов/мес

- **Архитектура:** Кластер с LB (Level 4)
- **Серверов:** 5–6
- **IP:** 2 публичных + 3–4 приватных
- **Домены:** ocp.company.com, studio.company.com, reports.company.com и др.

Роль	vCPU	RAM	SSD	HDD
App 1	8	32 GB	60 GB	50 GB
App 2	8	32 GB	60 GB	50 GB
DB Primary	16	64 GB	800 GB	50 GB
DB Replica	16	64 GB	800 GB	50 GB
S3/Logs	4	8 GB	50 GB	500 GB
Суммарно	52	200 GB	—	—

Нагрузка:

Метрика	Значение
Пиковый RPS	~450
WebSocket соединений	5 000–10 000
Операторов онлайн	до 1 000

Хранение (1 год):

Компонент	Объём	Тип
PostgreSQL	~104 GB	SSD
Логи	~150 GB	HDD
Файлы/S3	~150 GB	HDD

НА: Обязательно (DB Replication + LB)

При >250 000 диалогов/мес – обратитесь к команде разработки для индивидуального проектирования.

1. Обзор

Назначение

Документ описывает требования к серверной инфраструктуре для развёртывания Self-hosted Edition в зависимости от планируемой нагрузки (количества диалогов в месяц).

Диалог — полный цикл общения с клиентом, проходящий все стадии жизненного цикла: создание → назначение → обработка оператором/ботом → постобработка → закрытие. В среднем один диалог содержит ~11.5 сообщений.

Требования к ПО

Компонент	Версия
ОС	Ubuntu 22.04 LTS / RHEL 8+ / Debian 12+
Docker Engine	24.0+
Docker Compose	v2.20+
Nginx (reverse proxy)	1.24+
Certbot (Let's Encrypt)	2.0+ (опционально)

Поддерживаемые платформы виртуализации

VMware vSphere, Nutanix AHV, Proxmox VE, KVM/QEMU, Hyper-V, Bare Metal.

| GPU не требуется. LLM-инференс выполняется через внешние API-провайдеры.

2. Архитектура компонентов

Система построена на базе микросервисной архитектуры. Компоненты развёртываются через Docker-контейнеры и системные сервисы, управляются через `docker-compose.yml` и Ansible.

Основные слои:

- **Телефония** – SBC (Session Border Controller) – обработка SIP-трафика, IVR, маршрутизация вызовов, запись разговоров
 - **Бизнес-логика** – ОСР Backend + Proxy + File API – управление диалогами, очередями, операторами, файлами
 - **Фронтенд** – ОСР Frontend (React SPA) – операторская панель, обслуживается через Nginx
 - **База данных** – PostgreSQL (рекомендуется без контейнера в production)
 - **Файловое хранилище** – MinIO (S3-совместимое)
 - **Reverse Proxy** – Nginx – SSL termination, проксирование на все сервисы
 - **Отчётность** – Apache Superset – аналитические дашборды, встраиваемые в ОСР через iframe
 - **Мониторинг** – Grafana + Loki + Prometheus + Alloy – логи, метрики, алерты
-

3. Список компонентов и контейнеров

Платформа состоит из основных и опциональных компонентов. Основные компоненты управляются через `docker-compose.yml`.

Конкретные версии образов указаны в `docker-compose.yml` и могут обновляться между релизами.

Основные компоненты

#	Компонент	Тип	Порт	Доступ	Назначение
1	PostgreSQL	Системный сервис ¹	5432	internal	Основная СУБД для всех сервисов
2	Nginx	Системный сервис	80, 443	external	Reverse proxy, SSL termination
3	SBC	Docker-контейнер	5060, 5063	external (SIP/WSS)	Телефония: SIP-обработка, IVR, маршрутизация, запись
4	OCP Backend	Docker-контейнер	9999	internal (через Nginx)	Бизнес-логика: диалоги, очереди, операторы, API
5	OCP Frontend	Docker-контейнер / статика	8080	через Nginx	React SPA — операторская панель
6	OCP Proxy	Docker-контейнер	8888	internal (через Nginx)	Проксирование запросов между компонентами
7	OCP File API	Docker-контейнер	19999	internal (через Nginx)	Загрузка/скачивание файлов, работа с S3

8	OCP Migrations	Docker-контейнер	—	internal	Применение миграций БД (запускается при обновлении)
9	MinIO	Docker-контейнер	9000, 9001	internal	S3-совместимое файловое хранилище
10	Apache Superset	Docker-контейнер	8088	internal (через Nginx)	Аналитика и отчётность, встраивание через iframe

¹ Для production рекомендуется PostgreSQL как системный сервис (не в контейнере). Для тестовых инсталляций допустим контейнер.

Мониторинг

#	Компонент	Тип	Порт	Назначение
11	Grafana	Docker-контейнер	3000	Дашборды и визуализация метрик
12	Loki	Docker-контейнер	3100	Агрегация и хранение логов
13	Prometheus	Docker-контейнер	9090	Сбор и хранение метрик
14	Alloy	Docker-контейнер	—	Сбор логов с контейнеров (замена Fluent Bit)

Опциональные компоненты

#	Компонент	Тип	Порт	Назначение
---	-----------	-----	------	------------

| 16 | Screen Recorder | Docker-контейнер | — | Запись экранов операторов | | 17 | Video Streaming | Docker-контейнер (x2) | — | Видеостриминг и видеозапись | | 18 | Data Store (Storage API) | Docker-контейнер | — | Экспорт данных из OCP | | 19 | ClickHouse Sync | Docker-контейнер | — | Синхронизация данных в ClickHouse |

Потребление RAM по компонентам (типичное)

Компонент	Типичное	Пиковое	Примечание
PostgreSQL	1–2 GB	4–8 GB	shared_buffers = 25% RAM
SBC	500 MB	2 GB	зависит от кол-ва одновременных вызовов
OCP Backend	300 MB	1 GB	зависит от кол-ва операторов
OCP Proxy	100 MB	300 MB	
OCP File API	100 MB	300 MB	
OCP Frontend + Nginx	50 MB	100 MB	статика
MinIO	200 MB	500 MB	
Superset	300 MB	1 GB	зависит от сложности отчётов
Мониторинг (Grafana + Loki + Prometheus + Alloy)	400 MB	1 GB	
Итого (основные)	~3 GB	~8–15 GB	

4. Сетевая конфигурация

4.1. Домены

Для работы платформы необходим набор доменов. Все домены проксируются через Nginx.

Обязательные домены:

Домен	Назначение	Проксирование
ocp.company.com	OCP – операторская панель + API	Nginx → OCP Frontend :8080 / OCP Backend :9999
studio.company.com	Мастер-домен SBC	Nginx → SBC :5063 (WSS)
reports.company.com	Apache Superset – отчётность	Nginx → Superset :8088
softphone.company.com	CDN софт-фона (WebRTC)	Nginx → статика / SBC
monitoring.company.com	Grafana – мониторинг	Nginx → Grafana :3000

Дополнительные домены (при необходимости):

Домен	Назначение
client1.studio.company.com	Клиентские домены SBC
proxy.company.com	OCP Proxy (при отдельной установке)
data.company.com	Data Manager

| `storage.company.com` | S3-хранилище MinIO (или по IP) || `sbc-storage.company.com` | Доступ к записям разговоров через Nginx || `video.company.com` | Видеосервисы || `api.company.com` | Интеграционные сервисы |

Wildcard-сертификат `*.studio.company.com` *обязателен для работы WebRTC-клиентов.*

4.2. DNS-записи

<code>osp.company.com</code>	A	→ <IP App>
<code>studio.company.com</code>	A	→ <IP SBC>
<code>*.studio.company.com</code>	A	→ <IP SBC>
<code>reports.company.com</code>	A	→ <IP App>
<code>softphone.company.com</code>	A	→ <IP App>
<code>monitoring.company.com</code>	A	→ <IP App>

При разнесённой архитектуре (Level 2+) – БД-сервер не требует публичного DNS, доступ по приватному IP.

4.3. Таблица портов

Внешние порты (открыты в firewall):

Порт	Протокол	Назначение
80	TCP	HTTP → redirect на HTTPS
443	TCP	HTTPS (Nginx reverse proxy)
22	TCP	SSH (администрирование)
5060	UDP/TCP	SIP (телефония)
5063	TCP	SIP over TLS / WSS (WebRTC софт-фон)

Внутренние порты (только localhost / Docker network):

Порт	Сервис	Доступ
------	--------	--------

5432	PostgreSQL	Открыть только для DB-реплик
9999	OCP Backend	Через Nginx
19999	OCP File API	Через Nginx
8080	OCP Frontend	Через Nginx
8888	OCP Proxy	Через Nginx
9000	MinIO (S3 API)	Internal
9001	MinIO Console	Internal
8088	Superset	Через Nginx
3000	Grafana	Через Nginx
3100	Loki	Internal
9090	Prometheus	Internal

4.4. IP-адреса по конфигурациям

Конфигурация	Публичных IP	Приватных IP	Примечание
Pilot (1K)	1	0	Всё на одном сервере
Start (10K)	1	0	Всё на одном сервере
Standard (100K)	1	1	App (публичный) + DB (приватный)
Medium (200K)	1	2–3	App + DB Primary + S3/Logs (+ DB Replica)

Large (300K)	2	3–4	2× App + DB Primary + DB Replica + S3
--------------	---	-----	---------------------------------------

4.5. Firewall (UFW)

```
# Базовая конфигурация
ufw default deny incoming
ufw default allow outgoing

# SSH
ufw allow 22/tcp

# HTTP/HTTPS
ufw allow 80/tcp
ufw allow 443/tcp

# SIP ()
ufw allow 5060/udp
ufw allow 5060/tcp
ufw allow 5063/tcp

# PostgreSQL реплика (только для DB-сервера, только с App-)
# ufw allow from <APP_SERVER_IP> to any port 5432 proto tcp

ufw enable
```

Исходящий трафик

Контейнеры требуют исходящий HTTPS-доступ для работы с внешними сервисами:

Сервис	Адрес	Компонент
LLM API (OpenAI, Anthropic и др.)	api.openai.com:443 и аналогичные	OCP Backend
Telegram Bot API	api.telegram.org:443	OCP Backend
WhatsApp Business API	Зависит от провайдера	OCP Backend
SIP-провайдеры	Зависит от провайдера, порт 5060	SBC

SMTP-сервер	Настраивается, порт 587/465	OCP Backend
Docker Hub (обновления образов)	<code>registry-1.docker.io:443</code>	хост
Let's Encrypt (OCSP, CRL)	<code>acme-v02.api.letsencrypt.org:443</code>	хост
Производственный календарь	<code>production-calendar.ru:443</code>	OCP Backend

Если политика безопасности требует *whitelist* исходящих соединений, необходимо разрешить указанные адреса.

5. SSL-сертификаты

Варианты

Тип	Применение	Автопродление
Let's Encrypt (wildcard)	Dev, Staging, Small Prod	Да (certbot)
Let's Encrypt (отдельные)	Если нет доступа к DNS API	Да (certbot)
Корпоративный CA	Enterprise, банки, госсектор	Ручное
Self-signed	Только для dev/testing	—

Требования

- Wildcard сертификат `*.studio.company.com` покрывает мастер-домен SBC и все клиентские поддомены (обязателен для WebRTC)
- Отдельные сертификаты для `ocp.company.com`, `reports.company.com` и других доменов платформы
- Или единый wildcard `*.company.com` при использовании общего корневого домена
- Nginx выполняет SSL termination — внутренний трафик идёт по HTTP

Let's Encrypt с Certbot

```
# Wildcard для SBC (требует DNS challenge)
certbot certonly --manual --preferred-challenges dns \
  -d "*.studio.company.com" -d "studio.company.com"

# Отдельные сертификаты для остальных доменов (HTTP challenge)
certbot certonly --nginx -d ocp.company.com -d reports.company.com -d
monitoring.company.com
```

6. Конфигурации сайзинга

Базовые метрики для расчёта

Метрика	Значение	Источник
Средний размер диалога в БД	~16 KB	Анализ production-данных
Среднее кол-во сообщений/диалог	~11.5	Анализ production-данных
Таблицы на диалог	conversations, messages, chat_history, conversation_tags, chat_ratings, dialog_sessions, sla_events	Схема БД

Формула расчёта хранения

$$DB_size = dialogues_per_month \times 16 \text{ KB} \times months \times 1.5 \text{ (индексы)} \times 1.2 \text{ (WAL/bloat)}$$

Конфигурация Pilot — до 1 000 диалогов/мес

Один сервер, минимальная конфигурация для пилотных/пробных внедрений. Все компоненты на одной машине, HDD не требуется.

Параметр	Значение
vCPU	6–8
RAM	12–16 GB
SSD	100 GB (PostgreSQL data, Docker, OS, логи, файлы)

HDD	– (не требуется)
Пиковый RPS	~2–5
Concurrent WebSocket	~10–30
Операторов онлайн	до 5

Минимум для запуска: 6 vCPU / 12 GB RAM / 100 GB SSD. Рекомендуется 8 vCPU / 16 GB RAM для стабильной работы.

Расчёт хранения за 1 год:

Компонент	Объём
PostgreSQL (диалоги + индексы + WAL)	~0.35 GB
Логи (cron, API, webhook)	~0.5 GB
S3/файлы (вложения, аватары)	~0.5 GB
Docker images	~5 GB
OS + инструменты	~10 GB
Итого	~16 GB (всё на SSD)

Конфигурация Start — до 10 000 диалогов/мес

Один сервер, все компоненты на одной машине.

Параметр	Значение
vCPU	8
RAM	16 GB

SSD	100 GB (PostgreSQL data, Docker, OS)
HDD	50 GB (логи, бэкапы)
Пиковый RPS	~15
Concurrent WebSocket	~50–100
Операторов онлайн	до 20

Расчёт хранения за 1 год:

Компонент	Объём
PostgreSQL (диалоги + индексы + WAL)	~3 GB
Логи (cron, API, webhook)	~5 GB
S3/файлы (вложения, аватары)	~5 GB
Docker images	~5 GB
OS + инструменты	~10 GB
Итого	~28 GB (SSD: ~18 GB, HDD: ~10 GB)

Конфигурация Standard – 10 000 – 100 000 диалогов/мес

2 сервера: App + DB.

Параметр	App	DB
vCPU	8	8
RAM	32 GB	32 GB

SSD	60 GB	300 GB ¹
HDD	50 GB	100 GB

¹ Расчётное потребление PostgreSQL за первый год при 100К диалогов/мес — ~35 GB данных + ~17 GB индексы/WAL ≈ 52 GB. Рекомендуемый объём 300 GB включает запас на рост до 3 лет.

Метрика	Значение
Пиковый RPS	~150
Concurrent WebSocket	~500–2 000
Операторов онлайн	до 200

Расчёт хранения за 1 год (при 100К диалогов/мес):

Компонент	Объём
PostgreSQL (диалоги + индексы + WAL)	~35 GB
Логи (cron, API, webhook)	~50 GB
S3/файлы (вложения, аватары)	~50 GB
Docker images	~5 GB (×2 сервера)
OS + инструменты	~10 GB (×2 сервера)
Итого	~165 GB (распределено между серверами)

Конфигурация Medium — 100 000 – 200 000 диалогов/мес

3–4 сервера: App + DB Primary + S3/Logs (+ опционально DB Replica).

Роль	vCPU	RAM	SSD	HDD
------	------	-----	-----	-----

App (SBC, OCP Backend/Proxy/File API, Frontend, Nginx)	8	32 GB	60 GB	50 GB
DB Primary (PostgreSQL)	12	48 GB	500 GB	50 GB
S3/Logs (MinIO + Grafana/Loki/Prometheus)	4	8 GB	50 GB	300 GB
DB Replica (опционально, для HA)	12	48 GB	500 GB	50 GB

Метрика	Значение
Пиковый RPS	~300
Concurrent WebSocket	~2 000–5 000
Операторов онлайн	до 500

Расчёт хранения за 1 год (при 200К диалогов/мес):

Компонент	Объём
PostgreSQL (диалоги + индексы + WAL)	~69 GB
Логи (cron, API, webhook)	~100 GB
S3/файлы (вложения, аватары)	~100 GB
Docker images	~5 GB (×3–4 сервера)
OS + инструменты	~10 GB (×3–4 сервера)
Итого	~320 GB (распределено)

Конфигурация Large — 200 000 – 300 000 диалогов/мес

5–6 серверов: 2× App + DB Primary + DB Replica + S3/Logs. Load Balancer обязателен.

Роль	vCPU	RAM	SSD	HDD
App 1	8	32 GB	60 GB	50 GB
App 2	8	32 GB	60 GB	50 GB
DB Primary	16	64 GB	800 GB	50 GB
DB Replica	16	64 GB	800 GB	50 GB
S3/Logs (MinIO + Grafana/Loki/Prometheus)	4	8 GB	50 GB	500 GB

Метрика	Значение
Пиковый RPS	~450
Concurrent WebSocket	~5 000–10 000
Операторов онлайн	до 1 000

Расчёт хранения за 1 год (при 300К диалогов/мес):

Компонент	Объём
PostgreSQL (диалоги + индексы + WAL)	~104 GB
Логи (cron, API, webhook)	~150 GB
S3/файлы (вложения, аватары)	~150 GB
Docker images	~5 GB (×5–6 серверов)
OS + инструменты	~10 GB (×5–6 серверов)
Итого	~490 GB (распределено)

7. Дисковое пространство — детальный расчёт

7.1. PostgreSQL data (SSD обязательно!)

Формула:

$DB_size = (dialogues_per_month \times 16 \text{ KB} \times months) \times 1.5 \text{ (индексы)} \times 1.2 \text{ (WAL/bloat)}$

Пример для 100К диалогов/мес, 12 месяцев:

$100,000 \times 16 \text{ KB} \times 12 = 19.2 \text{ GB}$ (чистые данные)
 $\times 1.5$ (индексы, включая HNSW pgvector) = 28.8 GB
 $\times 1.2$ (WAL, bloat, vacuum) = ~34.5 GB
 + системные таблицы, конфиги: ~2 GB
Итого: ~35 GB /

Почему SSD: PostgreSQL с расширением pgvector активно использует random I/O для поиска по HNSW/IVFFlat индексам. На HDD латентность запросов RAG увеличивается в 5–10х.

7.2. Логи (HDD допустим)

Источник	Таблица/формат	Объём/год (10K)
Cron job logs	cron_job_logs	~500 MB/год
API logs	external_api_logs	~2 GB/год
Webhook logs	event_webhook_logs	~1 GB/год
Channel logs	external_channel_logs	~1 GB/год
Vector (Docker logs)	stdout/stderr всех контейнеров	~500 MB/год
Итого		~5 GB/год на 10K

Ротация логов:

Политика	Срок хранения	Применение
Минимальная	7 дней	Dev/Test
Стандартная	30 дней	Production
Расширенная	90 дней	Compliance, банки

7.3. S3 Storage (вложения, файлы)

Тип файла	Размер	Доля диалогов
Изображения (скриншоты)	200–500 KB	~15%
Документы (PDF, DOCX)	500 KB – 2 MB	~5%
Аватары операторов	50–100 KB	фиксированно
Экспорт отчётов	1–10 MB	по запросу

Формула:

$$S3_size = dialogues_per_month \times 0.15 \times 300 \text{ KB (avg image)}$$

$$+ dialogues_per_month \times 0.05 \times 1 \text{ MB (avg doc)}$$
Пример для 10К/мес:

$$10,000 \times 0.15 \times 300 \text{ KB} = 450 \text{ MB/мес } ()$$

$$10,000 \times 0.05 \times 1 \text{ MB} = 500 \text{ MB/мес } ()$$

Итого: ~1 GB/мес $\approx$ ~5 GB / 6 мес (с учётом компрессии)

7.4. Docker images + OS

Компонент	Объём
-----------	-------

Docker images (все контейнеры)	~5 GB
ОС (Ubuntu 22.04 minimal)	~5 GB
Системные утилиты, certbot, monitoring	~2–3 GB
Docker overlay2 (runtime)	~3–5 GB
Итого	~15–18 GB

7.5. Сводная таблица

Компонент	Тип	Start (10K)	Standard (100K)	Medium (200K)	Large (300K)
PostgreSQL data	SSD	20 GB	100 GB	200 GB	350 GB
PostgreSQL WAL/ temp	SSD	10 GB	40 GB	80 GB	120 GB
Docker images + OS	SSD	20 GB	20 GB ×N	20 GB ×N	20 GB ×N
Логи приложения	HDD	10 GB	50 GB	100 GB	150 GB
S3 / Вложения	HDD/S3	10 GB	50 GB	100 GB	150 GB
Бэкапы БД	HDD	10 GB	50 GB	100 GB	150 GB
Итого SSD (1 сервер)		50 GB	160 GB	300 GB	490 GB
Итого HDD		30 GB	150 GB	300 GB	450 GB

N = количество серверов в конфигурации. Объёмы SSD для Standard, Medium, Large относятся к DB-серверу.

8. Архитектура масштабирования

Level 1: All-in-one (1 сервер)

Конфигурации: Pilot, Start — до 10K диалогов/мес

Все компоненты на одном сервере: Nginx, SBC, OCP Backend, OCP Proxy, OCP File API, OCP Frontend, PostgreSQL, MinIO, Superset, мониторинг.

Level 2: App + DB (2 сервера)

Конфигурация: Standard — 10K – 100K диалогов/мес

- **App-сервер:** Nginx, SBC, OCP Backend, OCP Proxy, OCP File API, OCP Frontend, MinIO, Superset, мониторинг
- **DB-сервер:** PostgreSQL

Level 3: App + DB + S3/Logs (3–4 сервера)

Конфигурация: Medium — 100K – 200K диалогов/мес

- **App-сервер:** Nginx, SBC, OCP Backend, OCP Proxy, OCP File API, OCP Frontend, Superset
- **DB Primary:** PostgreSQL
- **S3/Logs:** MinIO, Grafana, Loki, Prometheus, Alloy
- **DB Replica (опционально):** PostgreSQL streaming replication

Level 4: Кластер (5–6 серверов)

Конфигурация: Large — 200K – 300K диалогов/мес, HA обязателен

- **App 1 + App 2:** Nginx + SBC + OCP Backend/Proxy/File API/Frontend за Load Balancer
- **DB Primary + DB Replica:** PostgreSQL streaming replication + автофейловер
- **S3/Logs:** MinIO, Grafana, Loki, Prometheus, Alloy

Компоненты HA:

Компонент	Технология
Load Balancer	HAProxy / Nginx Plus / Cloud LB

DB Failover	pg_auto_failover или Patroni
Session Affinity	Sticky sessions для WebSocket (SBC WebRTC)
Shared Storage	MinIO на отдельном сервере
SBC кластер	4 сервера SBC (требует поддержки SIP-провайдером)

9. Взаимодействие слоёв

9.1. Типичные потоки данных

Сценарий	Поток
Оператор открывает панель	Client → Nginx → OCP Frontend (SPA)
Загрузка списка диалогов	Client → Nginx → OCP Backend → PostgreSQL
Отправка сообщения	Client → Nginx → OCP Backend → PostgreSQL
Загрузка файла	Client → Nginx → OCP File API → MinIO
Входящий звонок	SIP-провайдер → SBC → IVR → Очередь → Оператор (через OCP)
Исходящий звонок	OCP Backend → SBC → SIP-провайдер
Входящий webhook (Telegram)	Telegram → Nginx → OCP Backend → PostgreSQL
AI-ответ бота	OCP Backend → External LLM API → OCP Backend → PostgreSQL
Прослушивание записи	Client → Nginx → sbc-storage (Nginx на SBC) → файл записи
Отчёт Superset	Client → Nginx → OCP (iframe) → Superset → PostgreSQL (read)
Авторизация через AD	Client → OCP Backend → AD Server (LDAP/LDAPS)

10. Резервное копирование

Стратегия

Метод	Назначение	Частота	Примечание
pg_dump	Логический бэкап БД	Ежедневно	Подходит для Start/Standard, полная переносимость
pg_basebackup	Физический бэкап	Еженедельно	Для Medium/Large, быстрое восстановление
WAL archiving	Непрерывная архивация WAL	Постоянно	Point-in-time recovery (PITR)
S3 sync	Файлы и вложения	Ежедневно	rclone sync или mc mirror

Размер бэкапов

Конфигурация	БД	S3/Файлы	Суммарно/год
Start (10K)	~3 GB	~1 GB	~7 GB
Standard (100K)	~35 GB	~12 GB	~84 GB
Medium (200K)	~69 GB	~23 GB	~161 GB
Large (300K)	~104 GB	~35 GB	~245 GB

RPO / RTO

Конфигурация	RPO	RTO	Метод
Start	24 часа	1–2 часа	pg_dump ежедневно
Standard	1 час	30 мин – 1 час	pg_dump + WAL archiving
Medium	~0 (минуты)	5–15 мин	Streaming Replication + PITR
Large	~0 (минуты)	5–15 мин	Streaming Replication + PITR + автофейловер

Пример cron для pg_dump

```
# /etc/cron.d/pg-backup
0 3 * * * root docker exec db pg_dump -U postgres -Fc postgres \
  | gzip > /backups/db_$(date +%Y%m%d).dump.gz

# Ротация: хранить последние 7
0 4 * * * root find /backups -name "db_*.dump.gz" -mtime +7 -delete
```

Куда складывать

Конфигурация	Основное хранилище	Дополнительно
Start	Локальный HDD раздел (/backups)	rsync на внешний сервер
Standard	Отдельный HDD раздел на DB-сервере	S3/MinIO на App-сервере
Medium	S3/MinIO сервер	Рекомендуется: geo-redundant storage
Large	S3/MinIO сервер	Обязательно: geo-redundant + offsite

11. Обновление системы

Процедура обновления

1. Создать бэкап БД перед обновлением (обязательно)
2. Остановить сервисы:
`docker compose down`
3. Обновить образы:
`docker compose pull`
4. Применить миграции БД (если есть):
`ansible-playbook -i inventory-remote.yml playbook-remote.yml --tags database`
5. Запустить сервисы:
`docker compose up -d`
6. Проверить здоровье всех контейнеров:
`docker ps --format "table {{.Names}}\t{{.Status}}"`

Downtime

Тип обновления	Время простоя	Примечание
Минорное (патч образов)	1–3 мин	<code>docker compose up -d</code> пересоздаёт изменённые контейнеры
Мажорное (миграции БД)	5–15 мин	Требуется остановки, применения миграций, перезапуска
PostgreSQL major version	30–60 мин	Требуется pg_upgrade или dump/restore

Рекомендации

- Обновление проводить в период минимальной нагрузки (ночь, выходные)
- Для конфигураций Medium/Large: обновлять App-серверы по одному (rolling update)

- Хранить предыдущие версии образов (`docker image tag`) для быстрого отката
-

12. Внешние зависимости и SMTP

SMTP (обязательно для production)

Для отправки email-уведомлений и подтверждений требуется SMTP-сервер.

Параметр	Переменная	Пример
Хост	SMTP_HOST	smtp.example.com
Порт	SMTP_PORT	587 (TLS) или 465 (SSL)
Пользователь	SMTP_USER	noreply@example.com
Пароль	SMTP_PASS	(через vault или prompt)
Имя отправителя	SMTP_SENDER_NAME	My App
Email администратора	SMTP_ADMIN_EMAIL	admin@example.com

Без настройки SMTP не будут работать: подтверждение email, восстановление пароля, уведомления операторам.

Внешние API-зависимости

Сервис	Контейнер	Обязательность	Назначение
LLM API (OpenAI, Anthropic и др.)	functions	Опционально	Для AI-ботов и RAG
Telegram Bot API	functions	Опционально	Для Telegram-канала
WhatsApp Business API	functions	Опционально	Для WhatsApp-канала

SMTP-сервер	auth	Обязательно	Email-уведомления
Docker Registry	хост	При обновлениях	Загрузка новых образов

При наличии жёстких политик безопасности (whitelist исходящих соединений) необходимо разрешить доступ к указанным API.

13. Чек-лист для DevOps

13.1. Пререквизиты

- ☐ Сервер(а) подготовлены согласно конфигурации (CPU, RAM, диски)
- ☐ ОС установлена: Ubuntu 22.04 LTS / RHEL 8+
- ☐ Docker Engine 24+ установлен
- ☐ Docker Compose v2+ установлен
- ☐ DNS-записи настроены для обоих доменов
- ☐ SSL-сертификаты получены (или настроен Certbot)
- ☐ SMTP-сервер настроен и доступен
- ☐ Firewall настроен (UFW: 22, 80, 443 + исходящий трафик)
- ☐ SSH-доступ по ключу (пароль отключён)
- ☐ NTP синхронизация включена
- ☐ Swap отключён или минимизирован (для PostgreSQL)

13.2. Порядок установки

1. Настройка ОС: обновление пакетов, установка Docker, firewall
2. SSL-сертификаты: получение и размещение
3. Конфигурация: заполнение переменных окружения (`group_vars/all.yml`)
4. Запуск Ansible:
`ansible-playbook -i inventory-remote.yml playbook-remote.yml`
5. Проверка здоровья: healthcheck всех 13 контейнеров
6. Первичная настройка: создание админа, настройка очередей, каналов
7. Настройка бэкапов: cron для pg_dump, ротация
8. Мониторинг: настройка сбора метрик (опционально)

13.3. Валидация после установки

Компонент	Команда / проверка
-----------	--------------------

Docker контейнеры	<code>docker ps --format "table {{.Names}}\n\t{{.Status}}"</code>
OCP Frontend	<code>https://ocp.company.com</code> – должна загрузиться SPA
OCP Backend	<code>curl -s https://ocp.company.com/api/health</code>
SBC	Подключение софт-фоном к <code>studio.company.com</code>
PostgreSQL	<code>docker exec -it db psql -U postgres -c "SELECT version();" или psql -U qms_db_user -d qms_db -c "SELECT 1;"</code>
MinIO	<code>curl -s http://<MINIO_IP>:9000/minio/health/live</code>
Superset	<code>https://reports.company.com</code> – должна загрузиться панель
SSL	<code>openssl s_client -connect ocp.company.com:443 -brief</code>
Записи разговоров	<code>curl -s http://sbc-storage.company.com/recordings/test.wav</code>
SMTP	Отправка тестового email через OCP

13.4. Мониторинг (рекомендации)

Метрика	Инструмент	Порог алерта
CPU usage	node_exporter + Prometheus	> 80% sustained

RAM usage	node_exporter	> 85%
Disk I/O	iostat / node_exporter	await > 20ms (SSD)
PostgreSQL connections	pg_stat_activity	> 80% max_connections
PostgreSQL replication lag	pg_stat_replication	> 1 MB
Docker container restarts	docker events	any restart
SSL cert expiry	certbot / blackbox_exporter	< 14 days

Приложение А: Сводная таблица конфигураций

Параметр	Pilot	Start	Standard	Medium	Large
Диалогов/мес	до 1K	до 10K	10K – 100K	100K – 200K	200K – 300K
Серверов	1	1	2	3–4 ¹	5–6
vCPU (суммарно)	6–8	8	16	24–36 ¹	52
RAM (суммарно)	12–16 GB	16 GB	64 GB	88–136 GB ¹	200 GB
SSD (суммарно)	100 GB	100 GB	360 GB	610–1 110 GB ¹	1 770 GB
HDD (суммарно)	—	50 GB	150 GB	400–450 GB ¹	700 GB
Публичных IP	1	1	1	1	2
Приватных IP	0	0	1	2–3	3–4
Пиковый RPS	~2–5	~15	~150	~300	~450
WebSocket	10–30	50–100	500–2 000	2 000–5 000	5 000–10 000
Операторов онлайн	до 5	до 20	до 200	до 500	до 1 000
HA	Нет	Нет	Опционально	Рекомендуется	Обязательно
PostgreSQL/год	~0.35 GB	~3 GB	~35 GB	~69 GB	~104 GB
Логи/год	~0.5 GB	~5 GB	~50 GB	~100 GB	~150 GB
S3/год	~0.5 GB	~5 GB	~50 GB	~100 GB	~150 GB

¹ Диапазоны для Medium указаны без/с DB Replica. Без реплики: 3 сервера, 24 vCPU, 88 GB RAM. С репликой (рекомендуется): 4 сервера, 36 vCPU, 136 GB RAM.

По вопросам масштабирования и нестандартных конфигураций обращайтесь к команде разработки.

План развёртывания платформы OCP

Общее описание

Данный документ содержит детальный план развёртывания платформы OCP (Operations Control Platform) с учетом всех зависимостей компонентов и правильной последовательности установки.

Стадия 1: Планирование и подготовка инфраструктуры

1.1 Проектирование сетевой архитектуры

Цель: Подготовка сетевой инфраструктуры для размещения компонентов.

Требования к сети:

- Прямая маршрутизация между SBC и провайдерами связи (без NAT)
- NAT может использоваться только при авторизованных транках
- Минимальные задержки между компонентами (желательно одна серверная стойка)

Действия:

1. Спроектировать подсеть для размещения всех компонентов
2. Настроить роутинг между SBC и провайдерами телефонии
3. Обеспечить отсутствие NAT для неавторизованных транков
4. Подготовить серверы согласно выбранной архитектуре

1.2 Выбор архитектуры развёртывания

Standalone (для пилотов и тестирования):

- 1 сервер: SBC + PostgreSQL
- Рекомендация: PostgreSQL без контейнера, SBC в контейнере

Кластерная (для продуктивной среды):

- 4 сервера SBC + 1 сервер PostgreSQL
- Альтернативно: кластерная PostgreSQL (Consul Patroni или PG Bouncer)
- Требует поддержки кластера SIP провайдером связи

1.3 Планирование доменов

Мастер-домен SBC (критически важно)

Роль и назначение:

- Корневой FQDN всей инсталляции Studio
- Фиксируется в коде активации и файле лицензии
- Хранит и распространяет общую лицензию, политики безопасности и настройки
- После инициализации изменение требует повторного выпуска лицензии

Требования к мастер-домену:

- Формат: `subdomain.company.tld` (для изолированных сетей допустимы зоны `.local`)
- DNS-резолвинг: запись должна существовать и корректно резолвиться до начала установки
- Стабильность: избегать временных проектов и персональных имен
- Корпоративность: использовать доменную зону компании
- Уникальность: проверить отсутствие конфликтов с существующими сервисами
- Масштабируемость: имя должно логично принимать иерархию поддоменов

Примеры правильных мастер-доменов:

<code>voice.mycompany.com</code>	# система телефонии
<code>uc.mycompany.com</code>	# unified communications
<code>studio.mycompany.local</code>	# для изолированной сети

Клиентские домены SBC

- Поддомены мастер-домена для конкретных клиентов, филиалов или бизнес-единиц
- Примеры: `client1.studio.mycompany.local`, `sales.voice.mycompany.com`
- Наследуют лицензионные ограничения от мастер-домена
- Получают часть ресурсов (DQL) через распределение лицензии
- Хранят конкретные настройки клиента: сценарии IVR, маршрутизацию, шаблоны автонастройки

Wildcard-сертификат (обязательно)

- **Требование:** Wildcard-сертификат `*.мастер-домен` обязателен для WebRTC
- **Назначение:** Покрывает мастер-домен и все клиентские поддомены
- **Критичность:** Без него браузеры блокируют HTTPS/WSS-соединения
- **Варианты получения:** Let's Encrypt (бесплатно, ACME, DNS-01) или коммерческий CA

Обязательные домены платформы

Домен	Назначение
<code>studio.company.com</code>	Мастер-домен SBC
<code>client1.studio.company.com</code>	Клиентские домены SBC
<code>ocp.company.com</code>	Домен OCP (обязательно с валидным сертификатом)
<code>softphone.company.com</code>	Домен для CDN софт-фона
<code>proxy.company.com</code>	Домен для прокси (при отдельной установке)
<code>data.company.com</code>	Домен для Data Manager

| `reports.company.com` | Домен для Superset | | `monitoring.company.com` | Домен для мониторинга |

Дополнительные домены (при необходимости)

Домен	Назначение
<code>storage.company.com</code>	S3-хранилище (или работа по IP)
<code>sbc-storage.company.com</code>	SBC записи разговоров (доступ через Nginx)
<code>video.company.com</code>	Видеосервисы
<code>api.company.com</code>	Интеграционные сервисы

Процедура утверждения доменов

1. Сформулировать список кандидатов доменных имен
2. Проверить DNS-свободность и FQDN-синтаксис
3. Согласовать с DNS-администратором и лицензиаром
4. Зафиксировать финальные имена в запросе на лицензию
5. Создать DNS-записи для всех доменов
6. Заказать wildcard-сертификат для мастер-домена
7. Настроить авто-продление сертификата

Важно: Изменение мастер-домена требует новой лицензии и перезаказа сертификата. Все клиентские домены должны быть поддоменами мастер-домена при `topleveldnpolicy = 1`.

1.4 Заказ лицензий SBC

Количественные параметры лицензии (DQL)

Базовые компоненты системы:

Параметр	Назначение
<code>domains</code>	Количество доменов. Расходуется по одному на каждый домен
<code>devices</code>	Количество зарегистрированных SIP-устройств
<code>siptrunks</code>	Количество одновременно активных транков
<code>sipqueue</code>	Количество ресурсов в hunt-группах с типом sip
<code>userqueue</code>	Количество ресурсов в hunt-группах с типом user

Сценарии и автоматизация:

Параметр	Назначение
----------	------------

<code>ivrscripts</code>	Количество экземпляров сценариев IVR
<code>svcscripts</code>	Количество экземпляров служебных сценариев
<code>chatbotscripts</code>	Количество экземпляров сценариев чат-ботов

Решения платформы (solution_):

Параметр	Назначение
<code>solution_sbc</code>	Количество солюшенов SBC
<code>solution_router</code>	Количество солюшенов маршрутизатора вызовов
<code>solution_bridge</code>	Количество солюшенов моста для интеграций
<code>solution_submaster</code>	Количество солюшенов подчиненных доменов
<code>solution_esb</code>	Количество солюшенов Enterprise Service Bus

Веб-сервисы и каналы интеграций:

Параметр	Назначение
<code>webservice_public</code>	Каналы интеграций с типом public
<code>webservice_subscr</code>	Каналы интеграций с типом subscr
<code>webservice_click-to-call</code>	Каналы «нажми и звони»
<code>webservice_crm_1c</code>	Интеграция с 1С
<code>webservice_crm_amo</code>	Интеграция с AmoCRM

<code>webservice_crm_b24</code>	Интеграция с Битрикс24
<code>webservice_crm_bpium</code>	Интеграция с Bpium
<code>webservice_crm_bpmsoft</code>	Интеграция с BPMSoft
<code>webservice_crm_creatio</code>	Интеграция с Creatio

Пример конфигурации лицензии

```
{
  "dql": {
    "devices": 400,
    "domains": 12,
    "ivrscripts": 26,
    "sipqueue": 90,
    "siptrunks": 1200,
    "solution_bridge": 12,
    "solution_esb": 2,
    "solution_router": 12,
    "solution_sbc": 2,
    "solution_submaster": 2,
    "svcscripts": 36,
    "webservice_public": 14,
    "webservice_subscr": 14
  },
  "smart": {
    "dtstop": "2025-09-31 23:59:59"
  }
}
```

Особенности распределения лицензий

- **Доменное дерево:** Лицензии распределяются по доменному дереву от мастер-домена к дочерним
- **Резервирование:** Каждый домен резервирует часть лицензий для внутреннего использования (ключ `licowned`)
- **Передача:** Домены передают лицензии дочерним доменам через свойство `lic`
- **Временные ограничения:** Поддержка суффикса `tilldt` для временного ограничения параметров
- **Автоматический пересчет:** Лицензия пересчитывается каждые несколько минут

Действия по заказу лицензии

1. Определить количество одновременных внешних разговоров (siptrunks)
2. Рассчитать количество внутренних пользователей (devices)
3. Спланировать количество очередей (sipqueue , userqueue)
4. Определить необходимое количество доменов (domains)
5. Рассчитать количество IVR и служебных сценариев (ivrscripts , svcscripts)
6. Выбрать необходимые решения платформы (solution_*)
7. Определить требуемые каналы интеграций (webservice_*)
8. Настроить качественные параметры (rec , cdr , sel , ap)
9. Подготовить заявку на лицензию через портал Oktell
10. Получить wildcard сертификат на мастер-домен

Важно: При нехватке лицензий система блокирует создание новых сущностей. Устройства проходят проверку через SIP OPTIONS перед отказом в регистрации.

Стадия 2: Установка базовых компонентов

2.1 Установка PostgreSQL

Порядок действий:

1. Установить PostgreSQL на выделенный сервер
2. Настроить базы данных для всех компонентов
3. Создать пользователей и настроить права доступа
4. Настроить подключения для всех сервисов

Сервисы, подключающиеся к PostgreSQL:

Основные компоненты:

Сервис	Назначение
SBC	Хранит конфигурации доменов, пользователей, маршрутизацию

OCP Backend	Основная база данных системы
OCP File API	Сервис для работы с файлами (та же БД что и OCP Backend)
Apache Superset	Система отчетности (подключается к OCP БД для чтения)

Дополнительные сервисы (опционально):

Сервис	Назначение
Data Store (Storage API)	Сервис экспорта данных

| Screen Recorder | Сервис записи экранов | | Синхронизация с ClickHouse | Перенос данных для аналитики | | Сервис миграций OCP | Обновления схемы БД | | Grafana | Хранение дашбордов и настроек |

Рекомендации по настройке БД:

- Создать отдельную БД для SBC
- Создать отдельную БД для OCP (основная)
- Создать отдельную БД для Superset (или использовать OCP БД в режиме чтения)
- Для тестовых инсталляций — одна PostgreSQL для всех сервисов
- Для продуктивных инсталляций — разделение БД по сервисам

Важно: PostgreSQL устанавливается первым, так как от него зависят все остальные компоненты.

2.2 Установка Nginx

Порядок действий:

1. Установить Nginx как системный сервис
2. Подготовить шаблоны конфигурации для всех доменов
3. Настроить SSL-сертификаты для всех доменов
4. Настроить проксирование для всех сервисов

Конфигурации:

- Базовая конфигурация для OCP
- Конфигурация для Superset
- Конфигурация для файлового сервиса

Стадия 3: Установка хранилища и мониторинга

3.1 Установка S3-хранилища (MinIO)

Порядок действий:

1. Развернуть MinIO контейнер
2. Создать административного пользователя
3. Создать бакет для системы
4. Получить ключи доступа (ACCESS_KEY , SECRET_KEY)
5. Настроить доступ по домену или IP

Важно: S3 должно быть установлено до ОСР, так как файловый сервис не сможет работать без него.

3.2 Установка системы мониторинга

Компоненты мониторинга:

- Grafana + Loki (для логов)
- Prometheus (для метрик)
- Alloy (вместо Fluent Bit, проверить поддержку JSON)

Порядок действий:

1. Развернуть Grafana и Loki
 2. Настроить сбор логов в JSON формате
 3. Установить Prometheus для метрик
 4. Настроить дашборды для мониторинга всех компонентов
-

Стадия 4: Установка и настройка SBC

4.1 Развертывание SBC

Для Standalone:

- Развернуть SBC контейнер на подготовленном сервере
- Использовать готовый конфиг для standalone версии

Для кластера:

- Развернуть 4 SBC сервера
- Использовать специальный кластерный конфиг
- Настроить кластерные функции с урезанными возможностями для совместимости с софтом

4.2 Первоначальная настройка SBC

API-плейбук настройки:

1. Создать административного пользователя (`qs-admin`)
2. Настроить права пользователя (включить «API-методы сценария выполнять»)
3. Создать SIP-устройство для административного пользователя
4. Настроить коды абонентских функций
5. Создать правила маршрутизации
6. Настроить мастер-домен
7. Настроить клиентский домен
8. Применить настройки клиентского домена

4.3 Установка патчей и настройка логотипов

Действия:

1. Обновить SBC с версии 2.2.3.1 до 2.4.1
2. Заменить логотипы в основном интерфейсе
3. Заменить логотипы в ролевых приложениях
4. Заменить логотипы в скрипт-редакторе
5. Применить патчи для корректной работы с софтом

4.4 Настройка записи разговоров и установка Nginx на SBC

Цель: Организовать доступ к аудиозаписям разговоров по HTTP-ссылкам.

Для Standalone установки:

1. Установить Nginx на SBC сервер как системный сервис
2. Настроить конфигурацию Nginx:

```
server {
    listen 80;
    server_name sbc-storage.company.com;

    location /recordings/ {
        alias /var/lib/oktell/recordings/;
    }
}
```

```

        autoindex off;
        add_header Content-Type audio/wav;
        add_header Cache-Control "no-cache, no-store, must-revalidate";
    }
}

```

1. Настроить права доступа к директории с записями
2. Проверить доступность записей по HTTP-ссылкам

Для кластерной установки:

1. Выбрать один сервер для хранения всех записей
2. Установить Nginx на выбранный сервер кластера SBC
3. Настроить централизованное хранение записей
4. В перспективе: реализовать микросервис для поиска записей на всех серверах кластера

Принцип работы:

1. SBC записывает разговоры в локальную директорию
2. Nginx шарит эту директорию через HTTP
3. OCP получает ссылки на записи и предоставляет их пользователям
4. Пользователи могут прослушивать записи через веб-интерфейс

Переменные окружения:

```
SBC_STORAGE=http://sbc-storage.company.com
```

Важно: В кластере записи хранятся только на одном сервере из-за технических ограничений. Необходимо предусмотреть ротацию и очистку старых записей.

4.5 Тестирование SBC

Для подключения СофтФона к SBC необходимо открыть доступ к порту 5063.

Проверки:

- [] Подключиться софт-фоном к SBC
- [] Протестировать входящие звонки через IVR
- [] Настроить переключение на внутренний номер в IVR
- [] Протестировать исходящие звонки
- [] Проверить появление записей в CDR
- [] Проверить доступность записей разговоров по ссылкам
- [] Убедиться в корректности отображения логотипов

4.6 Установка Apache Superset

Порядок действий:

1. Развернуть Superset контейнер
2. Подключить к PostgreSQL OCP для чтения данных
3. Настроить переменные окружения:

```
SUPERSET_DATABASE_URI=postgresql://qms_db_user:password@host:5432/qms_db
```

```
SUPERSET_SECRET_KEY=generate_secure_secret_key
```

```
SUPERSET_SERVICE_TOKEN=generate_service_token
```

1. Настроить административного пользователя Superset
2. Создать токен сервиса для API интеграции с OCP
3. Загрузить дашборды (процесс требует автоматизации)
4. Настроить права доступа к данным и отчетам

Настройка токена сервиса для встраивания iframe:

- **Назначение:** Токен обеспечивает безопасное встраивание отчетов Superset в iframe внутри OCP
 - **Генерация:** Токен должен быть сгенерирован с достаточной энтропией
 - **Встраивание:** OCP использует токен для генерации безопасных URL дашбордов
 - **Авторизация:** Пользователи видят отчеты без повторной аутентификации в Superset
-

Стадия 5: Установка дополнительных сервисов (опционально)

5.1 Установка Storage API сервиса

Описание: Backend сервис для экспорта данных из OCP.

Действия:

1. Развернуть Storage API контейнер
 2. Подключить к PostgreSQL OCP
 3. Настроить API для экспорта данных
 4. Настроить домен для внешнего доступа
-

Стадия 6: Развертывание OCP

6.1 Подготовка компонентов OCP

Компоненты системы:

- Frontend (статические файлы)
- Backend (основной сервис)
- Проxy сервис
- File API сервис
- Миграции базы данных

6.2 Установка OCP

Порядок действий:

1. Собрать frontend и разместить в директории Nginx
2. Развернуть backend контейнеры через docker-compose
3. Запустить миграции базы данных
4. Настроить переменные окружения согласно документации
5. Запустить все сервисы OCP

6.3 Переменные окружения OCP (.env)

База данных PostgreSQL (обязательно)

```
POSTGRES_DB=qms_db
POSTGRES_USER=qms_db_user
POSTGRES_PASSWORD=YGgpr8wwK3cU
POSTGRES_HOST=10.2.11.118
```

Хранилище S3/MinIO (обязательно)

```
BUCKET_NAME=ocp-bucket
S3_ACCESS_KEY=minioadmin
S3_SECRET_KEY=3N1sW5f68A9g4SI
S3_URL=http://10.2.11.116:9000
```

Интеграция с SBC (обязательно)

```
SBC_DATA_CHANNEL_TOKEN=440b63731fe1f06301d6aa1fe8f4259a
SBC_DOMAIN=studio.company.com
SBC_IVR_TOKEN=
SBC_LOGIN=qms_admin
SBC_PASSWORD=qms_admin
SBC=10.40.1.2
SBC_STORAGE=http://sbc-storage.company.com
```

Обязательно изменить `SBC_DATA_CHANNEL_TOKEN` и `SBC_IVR_TOKEN`.

Общие настройки системы

```
APP_ENV=DEV
API_KEY=ghJFj3123fdskjnvjhasjkyURY12Elkjnnakhoiuv331
LOG_LEVEL=INFO
LOG_JSON_FORMAT=True
CALENDAR_URL=https://production-calendar.ru/get-period
```

Система очередей и лицензирование

```
DELAY_LIC_DEVICES=60
QUEUES_WORKLOAD_PERIOD=2
```

Суперадминистратор (обязательно)

```
SUPERADMIN_LOGIN=admin
SUPERADMIN_PASSWORD=secure_password
```

Медиа-сервер (при использовании видео)

```
MEDIASERVER_IN_URL=rtsp://192.168.1.100:554
MEDIASERVER_OUT_URL=http://192.168.1.100:8080
```

Запись экранов (при необходимости)

```
SCREEN_RECORDER_DATA=config_data
SCREEN_RECORDING_ADMIN_LOGIN=admin
SCREEN_RECORDING_ADMIN_PASSWORD=password
```

Интеграция с Apache Superset (обязательно)

```
SUPERSET_DATABASE_URI=postgresql://qms_db_user:YGgpr8wwK3cU@10.2.11.118:5432/
qms_db
SUPERSET_SECRET_KEY=your_secret_key_here
SUPERSET_SERVICE_TOKEN=service_token_value
SUPERSET_URL=https://reports.company.com
```

Интеграция с Active Directory (опционально)

```
AD_DOMAIN=company.local
AD_SERVER=192.168.1.10
```

Фронтенд в режиме разработки (только для разработки)

```
VITE_PROXY_OCP_API=http://localhost:9999/
VITE_PROXY_FILE_API=http://localhost:19999/
```

Критически важно: Замените заглушки IP-адреса `0.0.0.0` и порты `0000` на реальные значения. Используйте безопасные пароли для всех компонентов.

6.4 Настройка интеграций

Подключение к SBC:

1. Настроить токены для DataChannel и IVR
2. Проверить подключение к SBC API
3. Настроить синхронизацию пользователей и лицензий

Подключение к хранилищу:

1. Настроить подключение к S3 (MinIO)
2. Проверить загрузку и скачивание файлов

3. Настроить загрузку аватаров пользователей

6.5 Настройка мониторинга OCP

1. Подключить все контейнеры OCP к системе мониторинга
2. Настроить сбор логов в JSON формате
3. Настроить алерты для критических ошибок
4. Проверить отображение метрик в Grafana

Стадия 7: Интеграция с внешними системами

7.1 Интеграция с Active Directory (при необходимости)

Принцип работы:

- OCP подключается к контроллеру домена для аутентификации пользователей
- Пользователи могут авторизовываться в OCP, используя учетные данные AD
- Синхронизация групп и ролей пользователей из Active Directory
- Автоматическое создание пользователей OCP при первом входе через AD

Требования к сети:

Порт	Назначение
389	LDAP
636	LDAPS
3268	Global Catalog

- DNS-резолвинг доменных имен контроллеров
- Синхронизация времени между OCP и контроллерами домена

Рекомендуется: использовать LDAPS (порт 636) для безопасного подключения. При недоступности AD система может переключиться на локальную аутентификацию.

7.3 Настройка видеозаписи (при необходимости)

Компоненты:

- 2 контейнера для видеостриминга и видеозаписи

- Клиент для записи экрана на компьютерах операторов

Действия:

1. Развернуть контейнеры видеосервиса
 2. Настроить интеграцию с S3 для хранения записей
 3. Установить клиенты записи экрана на рабочие места
-

Стадия 8: Финальная настройка и тестирование

8.1 Создание административных пользователей

1. Создать суперадминистратора в OCP
2. Настроить права доступа для администраторов
3. Создать тестовых пользователей для проверки функциональности

8.2 Комплексное тестирование

- ☐ Авторизация пользователей в OCP
- ☐ Подключение софт-фона к SBC через OCP
- ☐ Входящие и исходящие звонки
- ☐ Запись разговоров и их воспроизведение
- ☐ Загрузка файлов и аватаров
- ☐ Работа отчетов в Superset
- ☐ Функциональность мониторинга и логирования
- ☐ Интеграции с внешними системами

8.3 Оптимизация производительности

1. Проверить производительность всех компонентов
 2. Оптимизировать конфигурации при необходимости
 3. Настроить автоматическое резервное копирование
 4. Документировать финальные настройки
-

Порядок установки компонентов (краткая схема)

1. **PostgreSQL** — база данных для всех компонентов
2. **Nginx** — веб-сервер и прокси

3. **MinIO (S3)** – файловое хранилище
 4. **Grafana + Loki** – система мониторинга
 5. **SBC** – телефонная станция
 6. **Superset** – система отчетности
 7. **Data Store** – сервис экспорта данных
 8. **ОСП** – основная платформа
 9. **Интеграции** – внешние системы (AD, видео)
 10. **Тестирование** – комплексная проверка системы
-

Критически важные моменты

- **Зависимости компонентов:** Строго соблюдать порядок установки
- **Версии SBC:** Использовать только версию 2.4.1 или выше
- **Сетевая архитектура:** Избегать NAT между SBC и провайдерами
- **Мониторинг:** Устанавливать до развертывания основных сервисов
- **Безопасность:** Использовать SSL-сертификаты для всех доменов
- **Тестирование:** Тестировать каждый компонент перед переходом к следующему